Data Structures and Algorithms: Lab 1

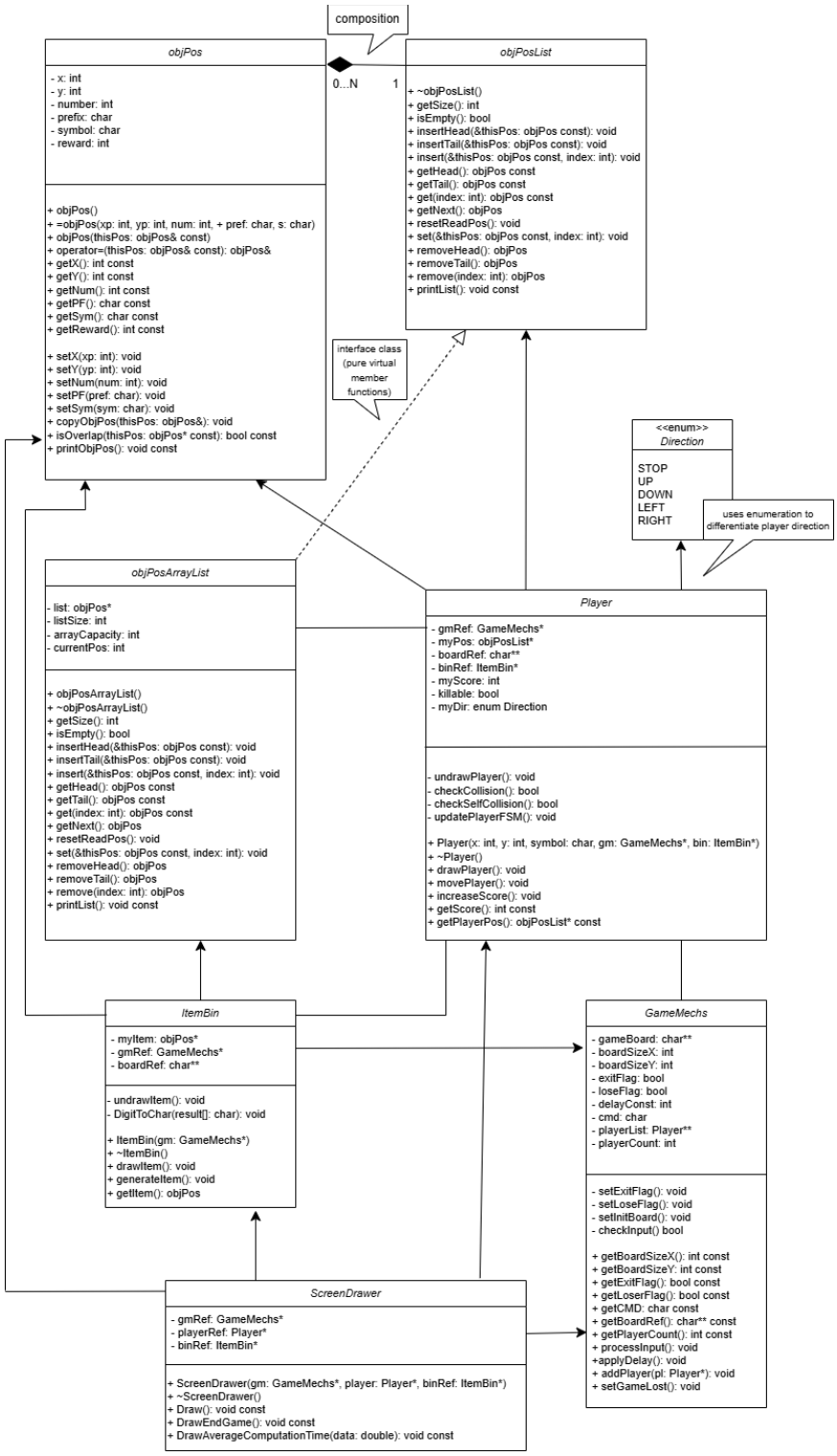
Time Complexity and Run Time (Asymptotic) Analysis

McMaster University

Christina Bridges

January 31, 2025

UML Diagram


composition
0...N
1
objPos
- x: int
- y: int
- number: int
- prefix: char
- symbol: char
- reward: int
+ objPos()
+ =objPos(xp: int, yp: int, num: int, + pref: char, s: char)
+ objPos(thisPos: objPos& const)
+ operator=(thisPos: objPos& const): objPos&
+ getX(): int const
+ getY(): int const
+ getNum(): int const
+ getPF(): char const
+ getSym(): char const
+ getReward(): int const
+ setX(xp: int): void
+ setY(yp: int): void
+ setNum(num: int): void
+ setPF(pref: char): void
+ setSym(sym: char): void
+ copyObjPos(thisPos: objPos&): void
+ isOverlap(thisPos: objPos* const): bool const
+ printObjPos(): void const
objPosList
+ ~objPosList()
+ getSize(): int
+ isEmpty(): bool
+ insertHead(&thisPos: objPos const): void
+ insertTail(&thisPos: objPos const): void
+ insert(&thisPos: objPos const, index: int): void
+ getHead(): objPos const
+ getTail(): objPos const
+ get(index: int): objPos const
+ getNext(): objPos
+ resetReadPos(): void
+ set(&thisPos: objPos const, index: int): void
+ removeHead(): objPos
+ removeTail(): objPos
+ remove(index: int): objPos
+ printList(): void const
interface class (pure virtual member functions)
<<enum>>
Direction
STOP
UP
DOWN
LEFT
RIGHT
uses enumeration to differentiate player direction
objPosArrayList
- list: objPos*
- listSize: int
- arrayCapacity: int
- currentPos: int
+ objPosArrayList()
+ ~objPosArrayList()
+ getSize(): int
+ isEmpty(): bool
+ insertHead(&thisPos: objPos const): void
+ insertTail(&thisPos: objPos const): void
+ insert(&thisPos: objPos const, index: int): void
+ getHead(): objPos const
+ getTail(): objPos const
+ get(index: int): objPos const
+ getNext(): objPos
+ resetReadPos(): void
+ set(&thisPos: objPos const, index: int): void
+ removeHead(): objPos
+ removeTail(): objPos
+ remove(index: int): objPos
+ printList(): void const
Player
- gmRef: GameMechs*
- myPos: objPosList*
- boardRef: char**
- binRef: ItemBin*
- myScore: int
- killable: bool
- myDir: enum Direction
- undrawPlayer(): void
- checkCollision(): bool
- checkSelfCollision(): bool
- updatePlayerFSM(): void
+ Player(x: int, y: int, symbol: char, gm: GameMechs*, bin: ItemBin*)
+ ~Player()
+ drawPlayer(): void
+ movePlayer(): void
+ increaseScore(): void
+ getScore(): int const
+ getPlayerPos(): objPosList* const
ItemBin
- myItem: objPos*
- gmRef: GameMechs*
- boardRef: char**
- undrawItem(): void
- DigitToChar(result[]: char): void
+ ItemBin(gm: GameMechs*)
+ ~ItemBin()
+ drawItem(): void
+ generateItem(): void
+ getItem(): objPos
GameMechs
- gameBoard: char**
- boardSizeX: int
- boardSizeY: int
- exitFlag: bool
- loseFlag: bool
- delayConst: int
- cmd: char
- playerList: Player**
- playerCount: int
- setExitFlag(): void
- setLoseFlag(): void
- setInitBoard(): void
- checkInput() bool
+ getBoardSizeX(): int const
+ getBoardSizeY: int const
+ getExitFlag(): bool const
+ getLoserFlag(): bool const
+ getCMD: char const
+ getBoardRef(): char** const
+ getPlayerCount(): int const
+ processInput(): void
+applyDelay(): void
+ addPlayer(pl: Player*): void
+ setGameLost(): void
ScreenDrawer
- gmRef: GameMechs*
- playerRef: Player*
- binRef: ItemBin*
+ ScreenDrawer(gm: GameMechs*, player: Player*, binRef: ItemBin*)
+ ~ScreenDrawer()
+ Draw(): void const
+ DrawEndGame(): void const
+ DrawAverageComputationTime(data: double): void const

Time Complexities

A. If boardSizeX = boardSizeY = n, then both the outer and inner for loop run n times. The statements on line 49 and 51 inside the *if* statement do not increase T(n) enough to alter the time complexity Θ as they only add T(c) to the run time where c is either the constant 2 or 3. The nested for loop configuration describes a time complexity of $\Theta(n^2)$.

```
42  void GameMechs::setInitBoard()  // private helper function
43  {
44      for(int i = 0; i < boardSizeY; i++)
45      {
46          for(int j = 0; j < boardSizeX; j++)
47          {
48              if(i == 0 || i == (boardSizeY - 1) || j == 0 || j == (boardSizeX - 1))
49                  gameBoard[i][j] = '#';
50              else
51                  gameBoard[i][j] = ' ';
52          }
53      }
54  }
```

B. Due to the return statement on line 34, $T(n) = \Theta_{best}(1)$ if the list size is equal to the value of the array capacity of the objPosArrayList. In contrast, if the return statement condition is not satisfied, the function will execute the remaining lines of code and in result $T(n) = \Theta_{worst}(n)$ because the for loop on line 38 must run n times while the remaining lines of code contribute a constant time operation. The algebraic expression *n* also describes the upper bound for O(n). Since *o* describes a weak upper bound and the time complexity cannot be n but only less than n, $o(n^2)$ describes a valid upper bound for the run time.

```
31  void objPosArrayList::insertHead(const objPos &thisPos)
32  {
33
34      if(listSize == arrayCapacity) return;
35
36      // The rest will be our estimation of Theta worst case
37
38      for(int i = listSize; i > 0; i--)
39      {
40          list[i] = list[i-1];  // can do this because of copy constructor
41      }
42
43      list[0].setX(thisPos.getX());
44      list[0].setY(thisPos.getY());
45      list[0].setNum(thisPos.getNum());
46      list[0].setPF(thisPos.getPF());
47      list[0].setSym(thisPos.getSym());
48
49      //list[0].printObjPos();
50
51      listSize++;
52  }
```

C. In the function objPosArrayList::removeTail() all statements, including the called copy constructor, constitute a constant time complexity and thus Θ(1).

```
objPos objPosArrayList::removeTail()
{
    if(listSize <= 0)
        return objPos(-99,0,0,0,0); // objPos constructor is called

    listSize--;  // lazy delete

    return list[listSize];  // return the lazy-deleted element
    // returning objPos instance through the stack will invoke copy constructor of objPos
}
```

D. In the function objPosArrayList::insert(), the first return statement after a constant time operation indicates $\Theta_{best}(1)$. If the condition on line 71 is not satisfied, the function will execute statements of a constant time complexity as well as of $\Theta(n)$ in the for loop as the elements shift in the array. The following statements also constitute a constant time complexity thus $\Theta_{worst}(n)$and $O(n)$ while $o(n^2)$ for the same reason as described in part B.

```
69  void objPosArrayList::insert(const objPos &thisPos, int index)
70  {
71      if(listSize == arrayCapacity) return;
72
73      if(index > listSize) index = listSize;
74      if(index < 0) index = 0;
75
76      for(int i = listSize; i > index; i--)
77      {
78          list[i] = list[i-1];
79      }
80
81      list[index].setX(thisPos.getX());
82      list[index].setY(thisPos.getY());
83      list[index].setNum(thisPos.getNum());
84      list[index].setPF(thisPos.getPF());
85      list[index].setSym(thisPos.getSym());
86
87      //list[index].printObjPos();
88
89      listSize++;
90  }
```

E. In the functions Player::drawPlayer() and Player::undrawPlayer(), the statements on lines prior to the for loop have a constant time complexity. The for loop has a time complexity of the array size, $\Theta(n)$, retrieved through the getSize() function on lines 167 and 181. The statements inside the for loop have a constant time complexity, thus the function's time complexity is $\Theta(n)$.

```
164  void Player::drawPlayer()
165  {
166      objPos targetPos;
167      int scanSize = myPos->getSize();  // get the list size
168
169      myPos->resetReadPos();
170      for(int i = 0; i < scanSize; i++)
171      {
172          targetPos = myPos->getNext();
173          boardRef[targetPos.getY()][targetPos.getX()] = targetPos.getSym();
174      }
175  }
176
177
178  void Player::undrawPlayer()  // private helper function
179  {
180      objPos targetPos;
181      int scanSize = myPos->getSize();  // get the list size
182
183      myPos->resetReadPos();
184      for(int i = 0; i < scanSize; i++)
185      {
186          targetPos = myPos->getNext();
187          boardRef[targetPos.getY()][targetPos.getX()] = ' ';
188      }
189  }
```

F. In the function ScreenDrawer::Draw(), lines 19 and 22 have a constant time complexity, however drawPlayer() is called and has a time complexity of $\Theta(n)$ due to the for loop. On lines 30 to 37, there is a nested for loop with a time complexity of $\Theta(n^2)$. Thus, $\Theta(n) + \Theta(n^2) = \Theta(n^2)$. Also, the lines in the function after the nested for loop exhibit a constant time complexity.

```
16  void ScreenDrawer::Draw() const
17  {
18      // Clear the Screen
19      MacUILib_clearScreen();
20
21      // redraw all items;
22      binRef->drawItem();
23      playerRef->drawPlayer();
24
25      // Get the Game Board 2D array
26      char** drawTarget = gmRef->getBoardRef();
27      objPos target = binRef->getItem();
28
29      // Draw it on the screen
30      for(int i = 0; i < gmRef->getBoardSizeY(); i++)
31      {
32          for(int j = 0; j < gmRef->getBoardSizeX(); j++)
33          {
34              MacUILib_printf("%c", drawTarget[i][j]);
35          }
36          MacUILib_printf("\n");
37      }
38
39      // Append any required debugging message below
40      MacUILib_printf("Player Score: %d\n", playerRef->getScore());
41      MacUILib_printf("Food Reward: %d\n", target.getReward());
42      //MacUILib_printf("Object: <%d, %d>, ID=%c%d\n", target.getX(), target.getY(), target.getP
43  }
```

```
164 void Player::drawPlayer()
165 {
166     objPos targetPos;
167     int scanSize = myPos->getSize();  // get the list size
168
169     myPos->resetReadPos();
170     for(int i = 0; i < scanSize; i++)
171     {
172         targetPos = myPos->getNext();
173         boardRef[targetPos.getY()][targetPos.getX()] = targetPos.getSym();
174     }
175 }
```

G. In the function Player::checkSelfCollision(), if the condition on line 143 is satisfied, then $\Theta(1) + \Theta(1) = \Theta_{best}(1)$ else due to the for loop of maximum iterations n, $\Theta_{worst}(n)$. Since the if statement in the for loop includes a return statement, the run time can be low compared to the run time for $\Theta_{worst}(n)$ and $O(n)$. As the strict upper bound is not known, a possible weak upper bound is $o(n^2)$. The reference function code is seen below:

```
139 bool Player::checkSelfCollision()  // private
140 {
141     // Make sure snake is long enough to kill itself
142     int length = myPos->getSize();
143     if(length < 4) return false;
144
145     // Then check for self collision
146     myPos->resetReadPos();
147     objPos tempPos;
148     objPos headPos = myPos->getNext();
149
150     for(int i = 1; i < length; i++)
151     {
152         tempPos = myPos->getNext();
153         if(headPos.isOverlap(&tempPos))
154         {
155             // set game end.
156             return true;
157         }
158     }
159
160     return false;
161 }
```

H. In the function Player::movePlayer(), the time complexity is O(n):

- Return statement has time complexity of $\Theta(1)$ on Line 70.
- undrawPlayer() is $\Theta(n)$ due to the for loop that iterates for the list size on line 184

- checkCollision() is assumed to return false thus the variable collided should be set to 0 on line 123 and won't satisfy the condition to enter into the for loop on line 126.
- Assuming the variable killable is false, checkSelfCollision() will never be executed during the run time, thus the final time complexity is O(n):

```
67  void Player::movePlayer()
68  {
69      updatePlayerFSM();   //1
70      if(myDir == STOP) return;    //1
71
72      undrawPlayer();  //n
73
74      objPos currHeadPos = myPos->getHead();   //1
75      int inX = currHeadPos.getX();  //1
76      int inY = currHeadPos.getY();   //1
77
78      switch(myDir) // constant
79      {
80          case UP:
81              if(--inY < 1)
82                  inY = gmRef->getBoardSizeY() - 2;  //1...
83              break;
84
85          case DOWN:
86              if(++inY > (gmRef->getBoardSizeY() - 2))
87                  inY = 1;
88              break;
89
90          case LEFT:
91              if(--inX < 1)
92                  inX = gmRef->getBoardSizeX() - 2;
93              break;
94
95          case RIGHT:
96              if(++inX > (gmRef->getBoardSizeX() - 2))
97                  inX = 1;
98              break;
99
100         default:
101             break;
102     }
103
104     currHeadPos.setX(inX);  //1
105     currHeadPos.setY(inY);// TARGET      //1
106
107     myPos->insertHead(currHeadPos);  // insert new head      //n
108
109     if(!checkCollision())    //n^2        // check collision.  If collision never happened,
110         myPos->removeTail();  //1       // removeTail.  Otherwise, generate new item.
111
112     if(killable)    //assume false
113         if(checkSelfCollision()) //n
114             gmRef->setGameLost();    //1   // If colliding with itself, end game.
115
116 }
```

```
bool Player::checkCollision()  // private
{
    // 1. Check Item Bin for Item ObjPos
    objPos target = binRef->getItem();
    bool collided = myPos->getHead().isOverlap(&target);  // returns 0

    // 2. check whether objPos are equal
    if(collided) // condition never entered if collided == 0
    {
        // 3. If yes, collision takes place
        // Generate New Item(s)
        binRef->generateItem(); //n^2

        // Increase Score by Number
        myScore += target.getNum();
    }

    return collided; // return assumed false value for part H
```

```
void Player::undrawPlayer()  // private helper function
{
    objPos targetPos;
    int scanSize = myPos->getSize();  // get the list size

    myPos->resetReadPos();
    for(int i = 0; i < scanSize; i++)
    {
        targetPos = myPos->getNext();
        boardRef[targetPos.getY()][targetPos.getX()] = ' ';
    }
}
```

I. The function ItemBin::generateItem() includes operations with a constant time complexity. At line 65, the function iterates through a nested for loop for the board size of m by m with $\Theta(m^2)$. The for loop on line 79 iterates for the length of the snake body thus has a time complexity of $\Theta(n)$. The do while loop on line 94 runs for at least one time and as long as the randomized candidates don't satisfy the while loop condition. Thus the final time complexity is $\Omega(n^2)$ since the complexity of the nested for loop sets a requirement of $\Theta(n^2)$ while the do while loop sets a lower bound. The time complexities for individual lines in the function are visible in the images below:

```cpp
void ItemBin::generateItem()
{
    // Step 1: Get Player Ref from GameMech for Player pos
    Player** plList = gmRef->getPlayerListRef();
    objPosList *playerPos = plList[0]->getPlayerPos();

    //int bitVec[gmRef->getBoardSizeX()][gmRef->getBoardSizeY()]= {0};

    // to prevent stack overflow
    int xsize = gmRef->getBoardSizeX();
    int ysize = gmRef->getBoardSizeY();
    int** bitVec = new int*[xsize];
    for(int i = 0; i < xsize; i++)     // Dr. Chen said to assume a board size of n x n
    {
        bitVec[i] = new int[ysize];
        for(int j = 0; j < ysize; j++)     // nested for loop --> theta(n^2)
            bitVec[i][j] = 0;
    }



    int playerLength = playerPos->getSize();    // theta(1)

    objPos target;
    playerPos->resetReadPos();     // theta(1)

    for(int i = 0; i < playerLength; i++)    // theta(n)
    {
        target = playerPos->getNext();
        bitVec[target.getX()][target.getY()] = 1;
    }

    int randCandidateX = 0;
    int randCandidateY = 0;
    int randCandidate = 0;

    // Step 2: Generate another food object
    // Coordinate Generation
    // x [2, BoardX-3]
    // y [2, BoardY-3]

    do
    {
        randCandidateX = rand() % (gmRef->getBoardSizeX() - 4) + 2;
        randCandidateY = rand() % (gmRef->getBoardSizeY() - 4) + 2;
    } while(bitVec[randCandidateX][randCandidateY] != 0);       // omega(1) on its own but with nested for loop is omega(n^2)
    //possibility that a position vector
    //corresponding to an empty space
    //is never randomized thus omega n^2
    //(bounded n^2 due to nested for loop above)


    undrawItem();  //theta(constant)

    myItem->setX(randCandidateX);  // theta(constant)
    myItem->setY(randCandidateY);

    // Prefix Generation
    // PF   [a-z, A-Z]
    randCandidate = rand() % 26 + 'A';  // 26 alpabets          //theta(constant)
    if(rand() % 2) randCandidate += 32; // randomly lowercased
    myItem->setPF((char)randCandidate);

    // Number Generation
    // Number [00, 99]
    myItem->setNum(rand() % 100);    // theta(constant)

    drawItem();  //theta(constant)

    for(int i = 0; i < xsize; i++)    //theta(n)
        delete[] bitVec[i];
    delete[] bitVec;
}
```